

A

Synopsis on

**AMASE: A Harness-Ablation Framework for  
Measuring Cost–Quality Trade-offs in Autonomous  
Software Engineering Agents**

*in partial fulfillment of the requirement for the degree*

of

Bachelor of Technology in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

**Aryan Raj Singh (<Roll No>)**

**<Team Member Name> (<Roll No>)**

**<Team Member Name> (<Roll No>)**

**<Team Member Name> (<Roll No>)**

Under the supervision of

Mr. <Supervisor Name> (Assistant Professor, CSE)



**NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY, GREATER  
NOIDA**

**Session 2026-27**

## Index

| <b>Sr.No.</b> | <b>Topics</b>                            | <b>Page No.</b> |
|---------------|------------------------------------------|-----------------|
| 1.            | Introduction                             | 1               |
| 2.            | Existing Systems                         | 6               |
| 3.            | Problem Statement                        | 9               |
| 4.            | Proposed Methodology                     | 11              |
| 5.            | Feasibility Study, Need and Significance | 19              |
| 6.            | Facilities Required for Proposed Work    | 22              |
| 7.            | Conclusion                               | 24              |
| 8.            | References                               | 26              |

Supervisor Sign:

(Sign of supervisor on synopsis is mandatory while submission)

# 1. Introduction

Autonomous software engineering agents are large language model (LLM) systems that do not merely generate code on request, but operate a closed perception–action loop over a real development environment. Such an agent reads files, searches a repository, edits source, executes shell commands, runs a test suite, observes the resulting standard output and standard error, and revises its own work until a stated goal is satisfied or a resource budget is exhausted. Between 2023 and 2026 this class of system moved from research prototype to routine industrial tooling, and the associated engineering discipline has shifted accordingly: the difficulty is no longer obtaining a capable base model, but constructing the software that surrounds it.

That surrounding software is called the agent harness, and it is the subject of this project. A harness comprises the tool interface exposed to the model, the system prompt and control flow, the strategy used to manage a finite context window, the mechanism that detects and escapes unproductive behaviour, the policy that decides which model serves which request, and the instrumentation that records what happened. None of these components involve retraining the model. All of them materially change what the resulting system can do.

The magnitude of that effect is the specific observation that motivates AMASE. Two independent 2026 sources report that harness design is worth between eleven and fifteen percentage points of task resolution on

established agent benchmarks while the underlying model is held constant. In one published account, a coding agent moved from a score of 52.8 to 66.5 on Terminal-Bench 2.0 — a gain of 13.7 points, sufficient to move the system from outside the top thirty to the top five of the leaderboard — with no change to the model whatsoever. An effect of this size is comparable to, and in several reported cases larger than, the improvement obtained by upgrading an entire model generation.

Despite this, the harness is largely unmeasured. Published agent work reports aggregate benchmark scores for complete systems, which conflates the contribution of the model with the contribution of the scaffolding around it. Individual harness mechanisms are described qualitatively in engineering blog posts and adopted by practitioners on the strength of anecdote. There is very little controlled, reproducible evidence attributing a specific effect size to a specific mechanism, and almost none that attaches a monetary cost to that effect. A mechanism that improves task resolution by two points while tripling token consumption is, for most deployments, a poor trade; the current literature does not permit that judgement to be made.

## **1.1 Project Overview**

AMASE (Autonomous Multi-Agent Software Engineering) is a harness-ablation framework. It consists of a deliberately minimal coding agent, a stack of eight independently switchable harness mechanisms, a sandboxed execution environment, a full telemetry pipeline, and an experiment runner that executes a configuration matrix with repetition. Its purpose is not to

produce a superior coding agent. Its purpose is to produce a measurement: for each harness mechanism, the marginal change in task resolution rate, token consumption, monetary cost per task, and ninety-fifth-percentile latency, reported with variance across repeated runs.

The central design decision follows directly from this purpose. The agent core is intentionally unsophisticated — a single ReAct-style reasoning-and-acting loop with seven tools. Every feature that a conventional project would treat as an architectural commitment is instead treated in AMASE as an experimental variable. Multi-agent decomposition, for instance, is not assumed to be beneficial; it is implemented as one switchable arm and tested against a single-agent baseline. The same applies to retrieval strategy, model routing, context compaction and reasoning-effort allocation.

## **1.2 Technology and Field**

The project lies at the intersection of applied artificial intelligence, software engineering and empirical systems evaluation. The implementation language is Python. Agent control flow is built on LangGraph, chosen for its checkpointing and deterministic replay facilities, which an ablation study requires. Model-generated code executes inside ephemeral Docker containers with the network disabled and hard limits on turns, wall-clock time and monetary spend; an equivalent interface to E2B Firecracker microVMs is provided for the multi-tenant threat model. Instrumentation follows the OpenTelemetry GenAI semantic conventions and exports to a

self-hosted Langfuse instance. Regression testing uses DeepEval within a GitHub Actions pipeline. Evaluation runs against the Terminal-Bench 2.x harness and against a decontaminated task set authored by the project team.

### **1.3 Technical Terminology**

- Agent harness — the tooling, prompting, control flow and context management surrounding a language model, excluding the model weights themselves.
- Ablation study — an experimental method in which individual components are removed or disabled in turn so that each component's contribution can be attributed separately.
- ReAct — a prompting and control paradigm that interleaves explicit reasoning traces with environment actions, enabling an agent to plan, act and revise within a single loop.
- Context rot — the empirically observed decline in a model's ability to retrieve and use information as the occupied context window grows, arising from the quadratic attention structure of transformers.
- Compaction — summarisation of accumulated interaction history when the context window approaches capacity, preserving decisions and unresolved issues while discarding redundant tool output.
- End-state verification — scoring an agent by executing tests against the final state of the environment, rather than by grading its transcript or its diff.

- Benchmark contamination — the presence of benchmark problems and their reference solutions in a model's training corpus, which inflates measured scores without corresponding capability.
- Semantic routing — profiling an incoming request and dispatching it to the least expensive model able to serve it at acceptable quality.

## 2. Existing Systems

Autonomous coding agents and their evaluation infrastructure constitute an active research area. This section surveys the systems, benchmarks and techniques that define the current state of practice, and identifies the gap that AMASE addresses.

### 2.1 Literature Survey

| Sr. | Work / System                                         | Year | Contribution                                                                                             | Limitation Relevant to This Project                                                             |
|-----|-------------------------------------------------------|------|----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| 1   | Yao et al., ReAct (arXiv:2210.03629)                  | 2022 | Interleaves reasoning traces with actions, establishing the control paradigm used by most coding agents. | Defines a loop; offers no account of which surrounding mechanisms make it reliable.             |
| 2   | Shinn et al., Reflexion (arXiv:2303.11366)            | 2023 | Verbal self-reflection over failed attempts as a form of reinforcement without weight updates.           | Gain reported for the technique as a whole; no cost accounting or component attribution.        |
| 3   | Jimenez et al., SWE-bench (arXiv:2310.06770)          | 2023 | First large-scale benchmark of real GitHub issue resolution; became the field standard.                  | Now saturated and contaminated; formally retired by OpenAI in 2026.                             |
| 4   | Yang et al., SWE-agent (arXiv:2405.15793)             | 2024 | Demonstrates that a purpose-built Agent-Computer Interface substantially raises resolution rates.        | Establishes that the interface matters, but does not decompose which interface elements matter. |
| 5   | Wang et al., OpenHands (arXiv:2407.16741)             | 2024 | Open platform for generalist software-development agents with a shared evaluation suite.                 | Optimises a complete product; individual scaffold mechanisms remain entangled.                  |
| 6   | Liu et al., Lost in the Middle (arXiv:2307.03172)     | 2023 | Shows retrieval accuracy degrades for information positioned mid-context.                                | Diagnoses the failure; does not evaluate engineering remedies in an agent loop.                 |
| 7   | Ong et al., RouteLLM (arXiv:2406.18665)               | 2024 | Learned routing between strong and weak models preserving ~95% quality at ~26% of cost.                  | Evaluated on chat benchmarks, not on long-horizon agentic software tasks.                       |
| 8   | Kwon et al., PagedAttention / vLLM (arXiv:2309.06180) | 2023 | Virtual-memory-style KV-cache management raising inference throughput substantially.                     | Serving-layer optimisation; orthogonal to harness-level decisions.                              |

| Sr. | Work / System                                              | Year | Contribution                                                                             | Limitation Relevant to This Project                                                         |
|-----|------------------------------------------------------------|------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| 9   | Zheng et al., MT-Bench & LLM-as-a-Judge (arXiv:2306.05685) | 2023 | Establishes model-based grading and quantifies its agreement with human judgement.       | Judge-based scoring is unreliable for code, where end-state execution is available.         |
| 10  | Badertdinov et al., SWE-rebench (arXiv:2505.20411)         | 2025 | Automated pipeline producing continuously refreshed, decontaminated SWE tasks.           | Supplies clean tasks but evaluates whole systems, not harness components.                   |
| 11  | Scale AI, SWE-Bench Pro (arXiv:2509.16941)                 | 2025 | 1,865 long-horizon tasks across four languages; ~107 lines changed over ~4.1 files.      | Raises task difficulty; leaves the attribution question untouched.                          |
| 12  | Merrill et al., Terminal-Bench (arXiv:2601.11868)          | 2026 | 89 Docker-sandboxed CLI tasks with end-state verification and a pluggable agent adapter. | Provides the measurement substrate this project builds upon; supplies tasks, not ablations. |

## 2.2 Analysis of Existing Approaches

Three observations follow from this survey. First, the field has convincingly established that the harness matters: SWE-agent attributes substantial gains to interface design, and 2026 industry reporting places scaffold choice at eleven to fifteen points. Second, no existing work isolates individual harness mechanisms under controlled conditions. Published results compare complete systems, so a practitioner cannot determine which of a dozen simultaneous design choices produced an observed improvement. Third, cost is almost universally omitted. Benchmarks report resolution rate; they do not report the tokens or currency expended to achieve it, even though that ratio governs whether a technique is deployable.

A fourth and more serious problem concerns measurement validity. In 2026 OpenAI formally withdrew SWE-bench Verified as a reported metric. An

audit of 138 of its problems found that 59.4 percent contained material defects: 35.5 percent enforced implementation details so strictly that functionally correct solutions were rejected, 18.8 percent tested behaviour never specified in the problem statement, and 5.1 percent exhibited other faults. The audit further documented direct training contamination, with frontier models reproducing reference patches verbatim and recalling inline comments from the original source. Any project that evaluates solely against such a benchmark inherits its invalidity.

## **2.3 Research Gap**

The gap this project occupies is therefore precise. The harness is known to be the dominant controllable factor in coding-agent performance; its individual mechanisms have not been measured separately; the cost of those mechanisms has not been reported; and the benchmarks conventionally used for such measurement are demonstrably compromised. AMASE addresses all four conditions simultaneously through controlled single-variable ablation, mandatory cost instrumentation, repeated runs with reported variance, and evaluation against a task set the project team authors and therefore knows to be uncontaminated.

### 3. Problem Statement

Practitioners building autonomous software engineering agents must choose among a growing catalogue of harness mechanisms — environment context injection, pre-completion verification, loop detection, context compaction strategies, sub-agent delegation, retrieval architecture, model routing and reasoning-effort allocation. These choices are currently made on the basis of anecdote, vendor documentation and imitation of published systems, because no controlled evidence exists attributing a measured effect size or a monetary cost to any individual mechanism.

This absence of evidence has three consequences. Engineering effort is misallocated toward mechanisms that are fashionable rather than effective, while cheap high-yield mechanisms are overlooked. Systems are deployed with unquantified unit economics, so that a technique improving resolution by a small margin at several times the token cost is adopted without that trade being visible. And because the benchmarks ordinarily used to settle such questions are saturated and contaminated — SWE-bench Verified having been retired by OpenAI in 2026 after an audit found 59.4 percent of sampled problems defective and documented verbatim reproduction of reference solutions — even well-intentioned comparisons rest on an unsound foundation.

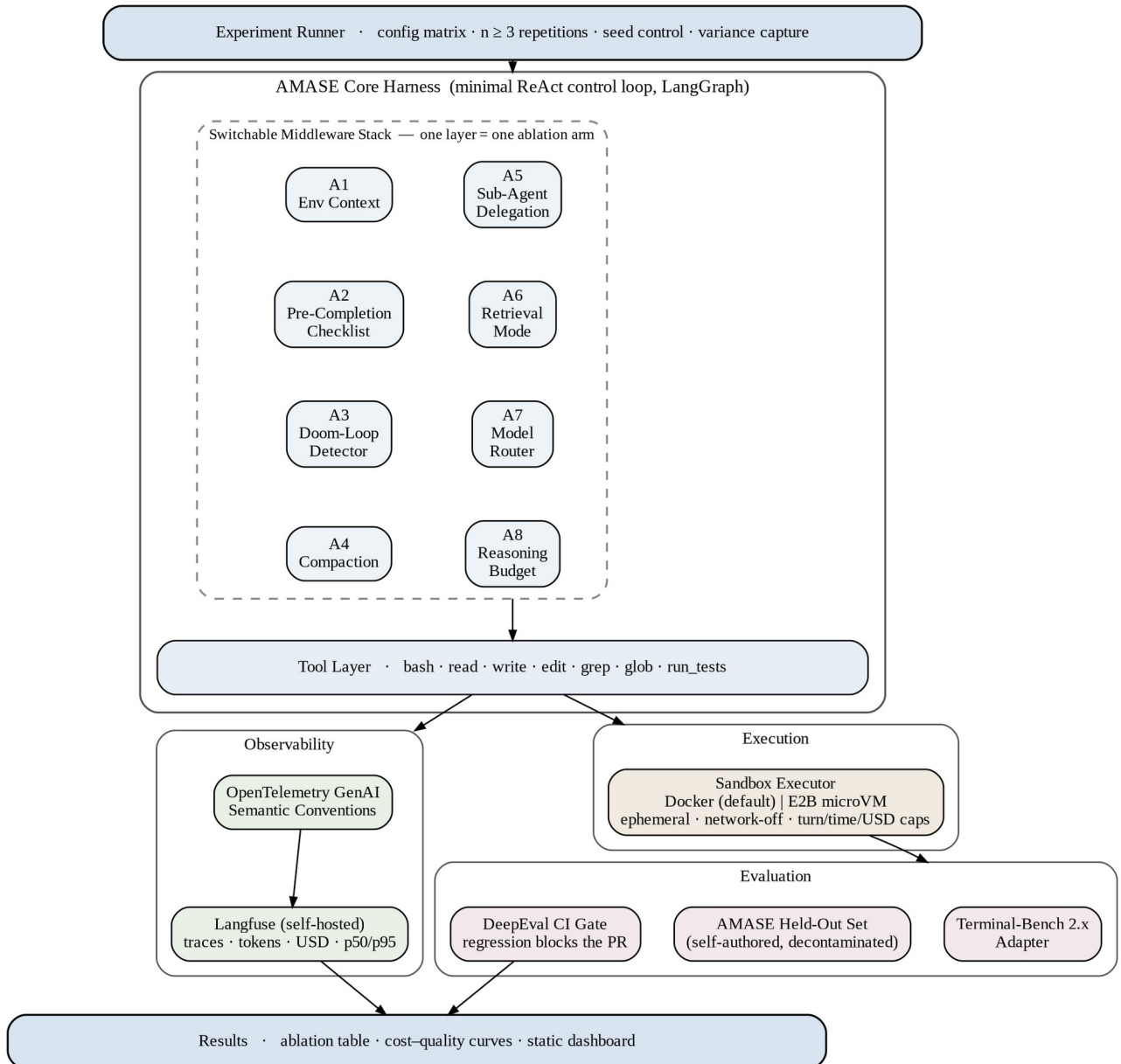
The problem this project addresses is thus the following: there exists no reproducible, cost-annotated, contamination-resistant experimental framework capable of attributing the performance of an autonomous

software engineering agent to its individual harness components. Consequently the single most influential controllable factor in such systems — worth an estimated eleven to fifteen percentage points with the model held fixed — remains, in the public literature, unquantified at the level of its constituent parts.

AMASE resolves this by constructing a minimal agent whose every harness mechanism is an independently switchable experimental arm, instrumenting every model call and tool invocation for tokens, cost and latency, executing each configuration a minimum of three times to permit variance reporting, and scoring outcomes by end-state verification against both a public benchmark and a decontaminated task set authored by the project team.

## 4. Proposed Methodology

The system is organised as seven cooperating modules. Modules 1 to 4 constitute the agent under test; modules 5 and 6 constitute the measurement apparatus; module 7 produces the research output. The separation is deliberate: the agent must be modifiable without disturbing the instrumentation that measures it.



**Figure 4.1: AMASE system architecture.**

## 4.1 Module 1 — Core Harness and Tool Layer

The core is a single ReAct control loop implemented on LangGraph, exposing seven tools to the model: `bash`, `read_file`, `write_file`, `edit_file`, `grep`, `glob` and `run_tests`. LangGraph is selected specifically for its checkpointing and state-persistence facilities, which allow a run to be replayed deterministically from an intermediate state — a prerequisite for controlled ablation. No middleware is active in the baseline configuration; every reported result is expressed as a delta from this baseline.

## 4.2 Module 2 — Switchable Middleware Stack

Eight mechanisms are implemented as independently toggleable middleware sharing a uniform interface. Each corresponds to one experimental arm.

| Arm | Mechanism                     | Function                                                                                                          | Hypothesis                                                                  |
|-----|-------------------------------|-------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| A1  | Environment context injection | Maps the directory tree, discovers installed tooling and states the turn and time budget at startup.              | Small positive effect at low token cost.                                    |
| A2  | Pre-completion checklist      | Blocks the completion signal until the agent has verified its output against the specification and run the tests. | Largest single positive effect.                                             |
| A3  | Doom-loop detection           | Tracks per-file edit counts and injects a forced re-plan when a threshold is crossed.                             | Reduces catastrophic runs; improves p95 latency.                            |
| A4  | Compaction strategy           | Selects among naive truncation, LLM summarisation and an external structured notes file.                          | Structured notes outperform summarisation on long-horizon tasks.            |
| A5  | Sub-agent delegation          | Runs review in a clean context window, returning a condensed summary to the coordinating agent.                   | Positive beyond a task-length threshold, negative below it.                 |
| A6  | Retrieval mode                | Selects among agentic search, hybrid BM25 with dense vectors and cross-encoder reranking, or both.                | Agentic search wins on exact symbols; hybrid wins on concept-level queries. |

| Arm | Mechanism                   | Function                                                                                                         | Hypothesis                                                            |
|-----|-----------------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| A7  | Model routing               | Applies static rules dispatching planning and debugging to a frontier model and mechanical edits to a local SLM. | Substantial cost reduction at small quality loss.                     |
| A8  | Reasoning budget allocation | Compares uniform effort against a high-plan, moderate-build, high-verify allocation.                             | Superior resolution per unit cost relative to uniform maximum effort. |

Arms A1, A2, A3 and A8 are inexpensive to implement and are expected to account for the majority of the total effect; they are scheduled first. Arms A6 and A7 are substantially more costly to build and are scheduled last. It is important to note that A6 and A7 encode claims frequently asserted as architectural necessities in industry writing; in AMASE they are treated strictly as hypotheses to be tested, and a negative result for either is a valid and publishable outcome.

### 4.3 Module 3 — Secure Execution Sandbox

Model-generated code is untrusted and is never executed on the host. Each task runs in an ephemeral Docker container with the network disabled, provisioned at the start of a rollout and destroyed on completion, with standard output and standard error captured and returned to the agent. Hard ceilings on turn count, wall-clock duration and cumulative monetary spend are enforced at this boundary, since an unbounded agent is both a correctness hazard and a budgetary one. A parallel implementation targeting E2B Firecracker microVMs is provided behind the same interface: Docker shares the host kernel, so for a genuinely multi-tenant deployment the dedicated-kernel isolation of a microVM is the correct choice, whereas for several hundred experimental rollouts local Docker is

the economically rational one.

## 4.4 Module 4 — Model Router and Retrieval Subsystem

The router profiles each outbound request and dispatches it to the least expensive model capable of serving it. Static rules based on request phase, token count and task-class keywords are implemented first, on the empirical observation that rule-based routing captures the majority of available savings at zero training cost; a learned classifier is introduced only if the static policy demonstrably plateaus. The retrieval subsystem implements both agentic search over grep, glob and direct file reads, and a hybrid pipeline combining BM25 sparse indexing, dense embeddings and cross-encoder reranking over AST-aware chunks, so that the two may be compared on identical tasks under identical conditions.

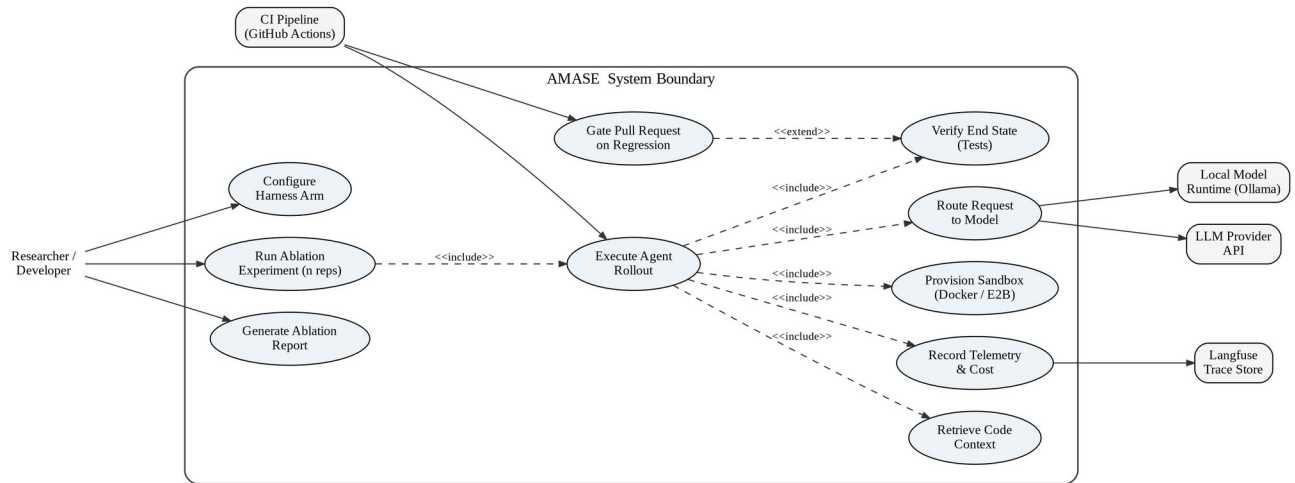
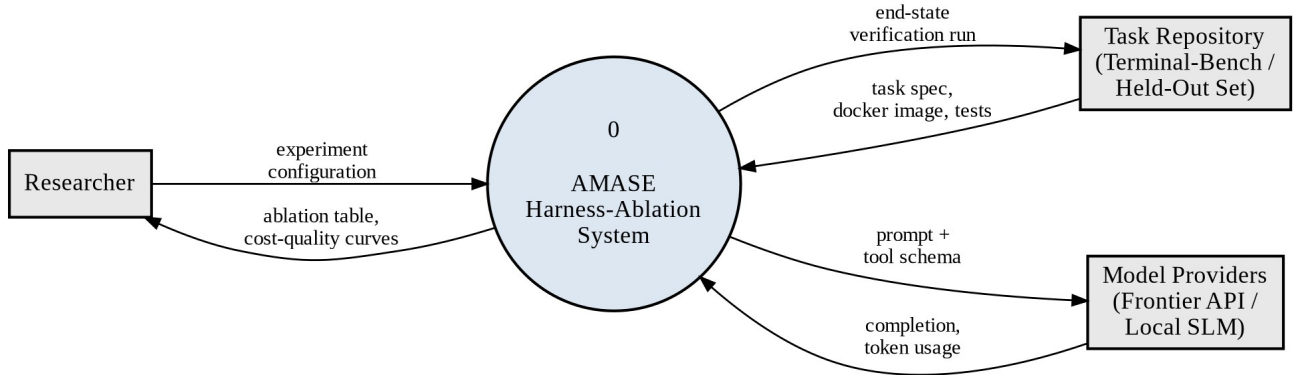
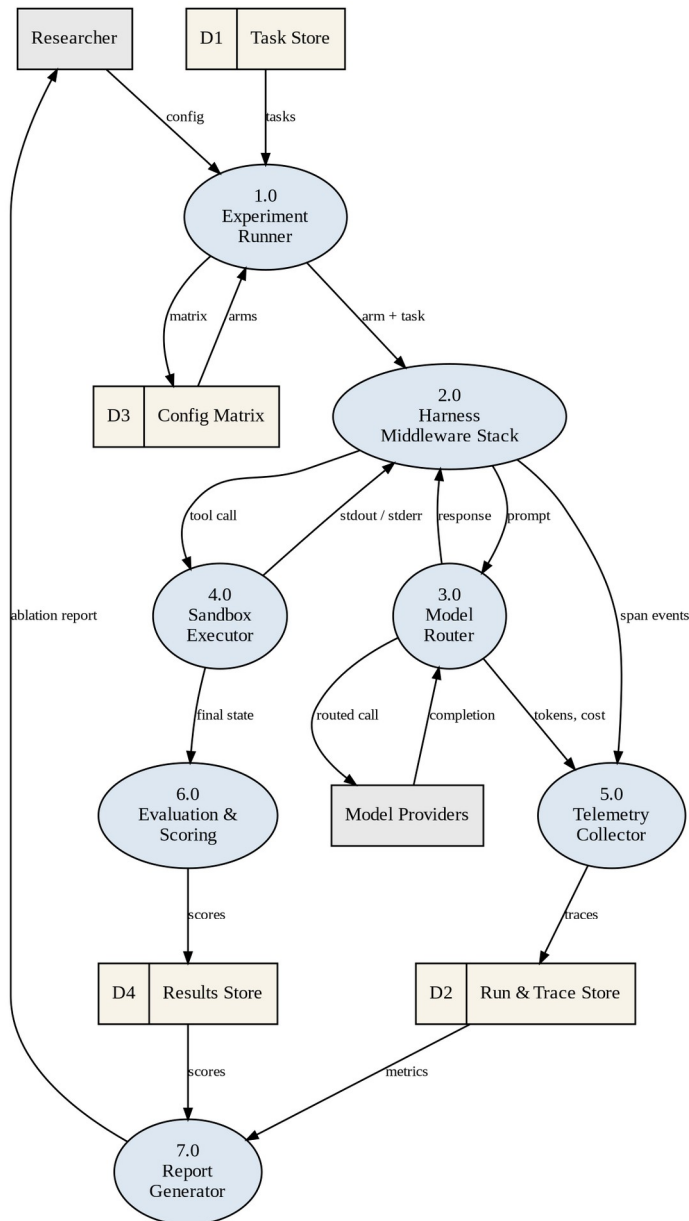


Figure 4.2: Use case diagram.



**Figure 4.3: Data flow diagram — Level 0 (context diagram).**



**Figure 4.4: Data flow diagram — Level 1.**

## 4.5 Module 5 — Telemetry and Observability

Every model call and every tool invocation emits a span conforming to the OpenTelemetry GenAI semantic conventions, exported to a self-hosted Langfuse instance. Conformance to an open standard rather than a vendor SDK is a deliberate choice, as it permits the backend to be replaced without reinstrumentation. The following quantities are recorded for every rollout and are treated as mandatory: resolution outcome; input and output tokens per model; monetary cost per model and per task; wall-clock duration at the fiftieth and ninety-fifth percentiles; turns to completion; tool-call error rate; doom-loop incidence; and, for retrieval comparisons, tokens per task and time to first relevant file. Every figure appearing in the final results is traceable to a stored span.

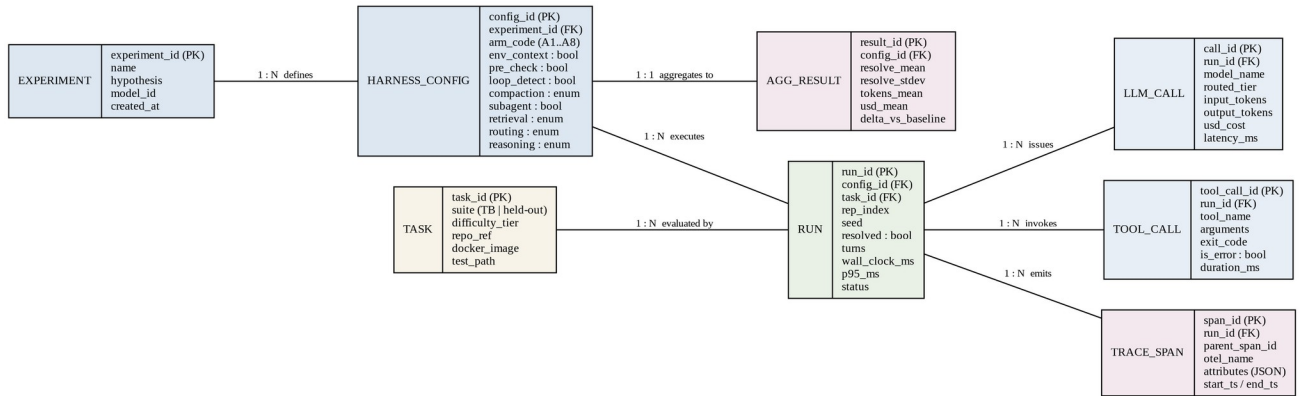


Figure 4.5: Entity-relationship diagram of the experiment and results schema.

## 4.6 Module 6 — Evaluation and Continuous Integration Gate

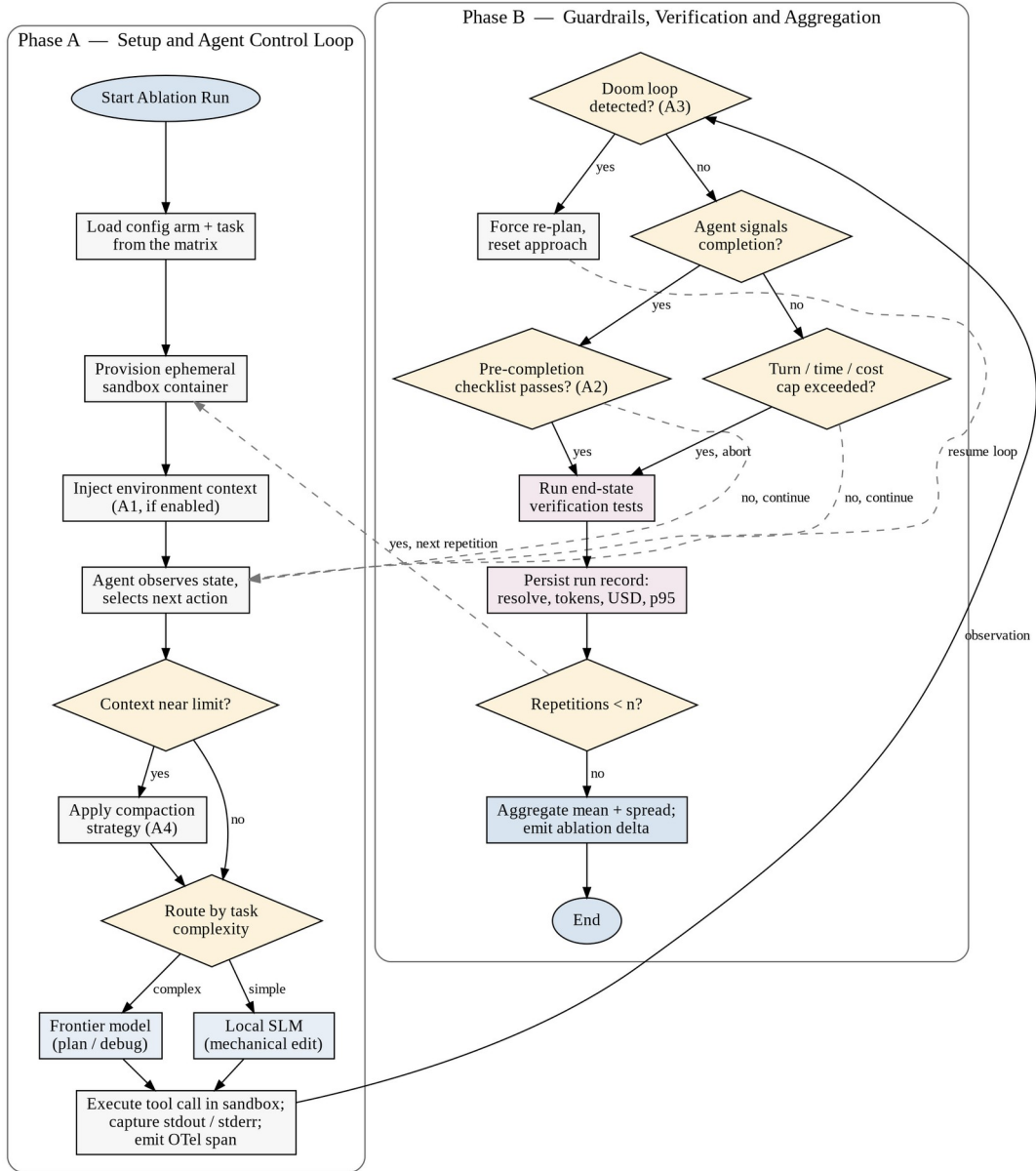
Two task sets are used. The first is Terminal-Bench 2.x, accessed through its adapter interface, which yields an externally comparable figure. The second, designated AMASE-Held-Out, comprises twenty-five to forty tasks authored by the project team in the Terminal-Bench format — a task

specification, a Dockerfile, a reference solution and a verification suite — drawn from repositories with no plausible presence in any training corpus. This second set is decontaminated by construction, and divergence between results on the two sets is itself an experimental finding. Scoring is by end-state verification: after the agent halts, the verification suite executes against the final environment state. Transcript grading is not used. Model-based grading is confined to the single case where no deterministic check exists, and its agreement with human judgement is reported.

A DeepEval suite executes within GitHub Actions on every pull request affecting prompts, middleware or configuration, running a reduced task subset against an inexpensive model and failing the request if resolution falls below the baseline by more than a defined tolerance, with the delta posted as a comment. The complete matrix executes nightly and its results are committed to the repository.

## **4.7 Module 7 — Experiment Runner and Reporting**

The runner executes a configuration matrix, varying a single mechanism at a time against the baseline and subsequently performing a reduced factorial over the three strongest mechanisms to detect interaction effects. Each configuration is executed a minimum of three times, since single-run agent benchmark figures are dominated by stochastic variation; mean and spread are reported throughout. Hypotheses are recorded in a pre-registration file before any run is executed. Cost is reported alongside every effect size without exception. Negative results are published rather than suppressed.



**Figure 4.6: Flowchart of a single ablation run.**

## **5. Feasibility Study, Need and Significance**

### **5.1 Technical Feasibility**

Every component required by AMASE exists as mature, documented and predominantly open-source software. LangGraph, Docker, Langfuse, DeepEval, Ollama and the Terminal-Bench harness are all publicly available and installable without special licensing; Terminal-Bench in particular is explicitly designed to accept third-party agents through an adapter interface, which removes the principal integration risk. The project requires no model training, no specialised accelerator hardware and no proprietary dataset. The engineering work consists of orchestration, instrumentation and disciplined experimental procedure, all of which fall within the competence of a final-year undergraduate team.

### **5.2 Economic Feasibility**

The dominant cost is inference. This is controlled by three measures. The full configuration matrix executes against an inexpensive hosted model or a locally hosted open-weight model served through Ollama, with frontier models reserved for a small number of final confirmatory runs. Hard per-rollout ceilings on turns, duration and spend prevent runaway consumption. Task counts are kept deliberately small, on the principle that twelve tasks executed three times yields more usable signal than forty tasks executed once. Cumulative expenditure is tracked within the same telemetry pipeline used for the experiments, making budget discipline itself observable. The projected total inference cost is modest and within the means of a student

team, and all supporting infrastructure is free and self-hosted.

### **5.3 Operational Feasibility**

The system is operated by its developers rather than by external end users, which removes adoption and training risk entirely. Execution is automated: a single command initiates a matrix run, and results are written as structured files committed to version control. The continuous integration gate is introduced early precisely so that subsequent experimental work is protected from silent regression.

### **5.4 Schedule Feasibility**

The work is decomposed into five phases with explicit exit criteria, and the phase ordering is chosen so that the project yields a defensible result even if later phases are curtailed. The measurement infrastructure — telemetry, cost accounting, the continuous integration gate and a baseline figure — is completed before any experimental arm is attempted. Should time prove insufficient for the more expensive arms, the project still delivers a fully instrumented, regression-protected agent harness with reported baseline performance.

### **5.5 Need for the Proposed Work**

Autonomous coding agents are being adopted in production while the basis for their design decisions remains anecdotal. Teams select harness mechanisms by imitation, cannot justify those selections quantitatively, and operate without knowledge of the unit economics of the systems they

deploy. The conventional means of settling such questions — public benchmarking — has been undermined by saturation and contamination severe enough that a major laboratory has withdrawn its own reported metric. A reproducible experimental framework that attributes effect and cost to individual mechanisms, and that evaluates against tasks known to be uncontaminated, is therefore needed.

## **5.6 Significance**

The contribution of this project is methodological as much as it is technical. It produces an attribution of performance to individual harness mechanisms with cost attached, a reusable experimental apparatus for extending that attribution to mechanisms proposed in future, and a decontaminated task set constructed to remain valid as public benchmarks continue to degrade. For practitioners, the outcome is direct guidance on where harness engineering effort is best spent. For the academic community, it offers a template for controlled evaluation of agentic systems in place of aggregate leaderboard comparison. The negative results are of comparable value to the positive ones, since demonstrating that a widely assumed technique does not pay for itself on a given task class is a substantive finding.

## 6. Facilities Required for Proposed Work

### 6.1 Hardware Requirements

| Component | Minimum Specification                                  | Purpose                                                    |
|-----------|--------------------------------------------------------|------------------------------------------------------------|
| Processor | Intel Core i5 (8th generation) or AMD Ryzen 5, 4 cores | Container orchestration and agent control loop             |
| Memory    | 16 GB RAM (32 GB recommended)                          | Concurrent Docker sandboxes and the Langfuse stack         |
| Storage   | 256 GB SSD with at least 80 GB free                    | Container images, task repositories, trace and result data |
| Graphics  | NVIDIA GPU with 8 GB VRAM (optional)                   | Local open-weight model inference for routing experiments  |
| Network   | Stable broadband connection                            | Model API access and container image retrieval             |

The optional GPU is required only for the local small-language-model arm of the routing experiment. In its absence that arm may be executed against a low-cost hosted model, with the substitution documented in the results.

### 6.2 Software Requirements

| Category             | Software                                  | Role in the System                                    |
|----------------------|-------------------------------------------|-------------------------------------------------------|
| Operating System     | Ubuntu 22.04 LTS or Windows 11 with WSL2  | Development and execution environment                 |
| Language and Runtime | Python 3.11 or later, uv package manager  | Primary implementation language                       |
| Agent Orchestration  | LangGraph                                 | Stateful control loop with checkpointing and replay   |
| Containerisation     | Docker Engine, Docker Compose             | Ephemeral execution sandboxes and supporting services |
| Alternative Sandbox  | E2B SDK (Firecracker microVM)             | Dedicated-kernel isolation for the multi-tenant case  |
| Observability        | OpenTelemetry SDK, Langfuse (self-hosted) | Span emission and trace storage                       |
| Evaluation           | Terminal-Bench 2.x, DeepEval, pytest      | Task execution, CI gating, end-state verification     |

| Category               | Software                                              | Role in the System                                      |
|------------------------|-------------------------------------------------------|---------------------------------------------------------|
| Retrieval              | Qdrant, rank_bm25, sentence-transformers, tree-sitter | Hybrid retrieval arm and AST-aware chunking             |
| Local Inference        | Ollama, vLLM                                          | Open-weight model serving for the routing arm           |
| Version Control and CI | Git, GitHub, GitHub Actions                           | Source control and the regression gate                  |
| Analysis and Reporting | pandas, matplotlib                                    | Aggregation, variance computation and figure generation |

### 6.3 Data Requirements

Two task collections are required. The Terminal-Bench 2.x public dataset is obtained through the official distribution. The AMASE-Held-Out collection is authored by the project team from repositories with no plausible training-corpus presence, comprising a task specification, a Dockerfile, a reference solution and a verification suite for each task, organised into three difficulty tiers. Tasks that the baseline configuration resolves in all repetitions or in none are discarded, as they carry no discriminative signal.

## 7. Conclusion

This synopsis has proposed AMASE, a harness-ablation framework for autonomous software engineering agents. The work is motivated by a specific and well-evidenced observation: the harness surrounding a language model is worth an estimated eleven to fifteen percentage points of task resolution with the model held constant, yet its individual mechanisms have never been measured separately, their costs have not been reported, and the benchmarks ordinarily used for such measurement have been shown to be saturated and contaminated.

The proposed system addresses these conditions directly. A deliberately minimal agent core ensures that no mechanism is assumed rather than tested. Eight independently switchable middleware layers permit single-variable ablation. Comprehensive telemetry conforming to an open standard attaches token and monetary cost to every measured effect. Repetition with reported variance guards against the stochastic noise that renders single-run agent benchmarking unreliable. A continuous integration gate protects accumulated results from regression. Evaluation against both a public benchmark and a team-authored decontaminated task set ensures that findings survive scrutiny of benchmark validity.

The expected outcome is an attribution table stating, for each of eight harness mechanisms, its marginal contribution to resolution rate together with its token and monetary cost, accompanied by cost–quality curves for the routing and retrieval arms and a reproducible apparatus permitting the

study to be extended. Negative findings are treated as results of equal standing: establishing that a widely assumed technique fails to justify its cost on a defined task class is a substantive contribution to a literature currently dominated by untested assertion.

In its scope the project is deliberately narrow and, within that scope, rigorous. It does not attempt to compete with established agent platforms on capability. It attempts instead to supply the controlled, cost-annotated evidence that the construction of such platforms currently lacks, and to demonstrate that disciplined empirical method remains applicable to systems whose behaviour is stochastic.

## 8. References

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models”, arXiv preprint arXiv:2210.03629, 2022.
- [2] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning”, arXiv preprint arXiv:2303.11366, 2023.
- [3] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, K. Narasimhan, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?”, arXiv preprint arXiv:2310.06770, 2023.
- [4] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, O. Press, “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering”, in Advances in Neural Information Processing Systems (NeurIPS), 2024. arXiv:2405.15793.
- [5] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, et al., “OpenHands: An Open Platform for AI Software Developers as Generalist Agents”, in International Conference on Learning Representations (ICLR), 2025. arXiv:2407.16741.
- [6] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang, “Lost in the Middle: How Language Models Use Long Contexts”, Transactions of the Association for Computational Linguistics, vol. 12, 2024. arXiv:2307.03172.
- [7] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, I. Stoica, “RouteLLM: Learning to Route LLMs with Preference Data”, arXiv preprint arXiv:2406.18665, 2024.
- [8] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, I. Stoica, “Efficient Memory Management for Large Language Model Serving with PagedAttention”, in Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP), 2023. arXiv:2309.06180.
- [9] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena”, in Advances in Neural Information Processing Systems (NeurIPS), 2023. arXiv:2306.05685.
- [10] I. Badertdinov, A. Golubev, M. Nekrashevich, et al., “SWE-rebench: An Automated Pipeline for Task Collection and Decontaminated Evaluation of Software Engineering Agents”, in Advances in Neural Information Processing Systems (NeurIPS), 2025. arXiv:2505.20411.
- [11] Scale AI, “SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?”, arXiv preprint arXiv:2509.16941, 2025.
- [12] M. A. Merrill, A. G. Shaw, N. Carlini, B. Li, H. Raj, et al., “Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces”, arXiv preprint arXiv:2601.11868, 2026.
- [13] OpenAI, “Why SWE-bench Verified No Longer Measures Frontier Coding Capabilities”,

OpenAI Technical Report, 2026. [Online]. Available: <https://openai.com/index/why-we-no-longer-evaluate-swe-bench-verified/>

- [14] Anthropic, “Effective Context Engineering for AI Agents”, Anthropic Engineering Blog, 2025. [Online]. Available: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
- [15] LangChain, “Improving Deep Agents with Harness Engineering”, LangChain Engineering Blog, 2026. [Online]. Available: <https://www.langchain.com/blog/improving-deep-agents-with-harness-engineering>